3.9.5、轮廓层次

[OpenCV：等高线层级]
https://docs.opencv.org/4.x/d9/d8b/tutorial_py_contours_hierarchy.html

前面我们学了找轮廓、轮廓特征、属性、更多函数，现在 Hierarchy 告诉你“轮廓之间是谁包含谁”的关系。为什么需要 Hierarchy（层次结构）？在现实图像中，物体经常是嵌套的：

- 一个大正方形里面有一个小正方形（小的是孩子的轮廓）
- 一个物体里面有“洞”（比如字母“O”、甜甜圈、手环）
- 复杂场景中可能有多层嵌套（爷爷 → 父亲 → 儿子）
  如果不使用 Hierarchy，所有轮廓都是“平级的”，你无法知道哪个在里面、哪个在外面。
  cv.findContours() 的第 3 个参数就是 retrieval mode（检索模式），它决定如何组织这些轮廓之间的关系。findContours 的完整调用方式

```
contours, hierarchy = cv.findContours(thresh, mode, method)
```

- mode：就是我们今天要学的 4 种模式（最重要）
- method：通常用 cv.CHAIN_APPROX_SIMPLE（压缩轮廓点）

1. cv.RETR_LIST（最简单，默认常用）

- 取出所有轮廓。
- 不建立任何父子关系，所有轮廓都是平级的。
- Hierarchy 中只有 Next 和 Previous 有意义，First_Child 和 Parent 永远是 -1。
- 适用场景：你只想拿到所有轮廓，不关心谁在谁里面（最常用入门模式）。

2. cv.RETR_EXTERNAL（只取最外层）

- 只取出最外层的轮廓（外部边界）。
- 所有内部的轮廓（洞、嵌套物体）都被忽略。
- 适用场景：只想检测物体的整体外轮廓，不想看到里面的洞或小物体（比如检测硬币、车辆外形）。

3. cv.RETR_CCOMP（两层层次结构）

- 取出所有轮廓，但只建立两层关系：
  - 第 1 层（Hierarchy-1）：外部物体边界（external contours）
  - 第 2 层（Hierarchy-2）：物体内部的“洞”（holes）

- 如果洞里面还有物体，它会被当成新的第 1 层。
- 适用场景：需要区分“物体”和“物体里的洞”，但不需要更深的嵌套。

4. cv.RETR_TREE（完整树结构，最强大）

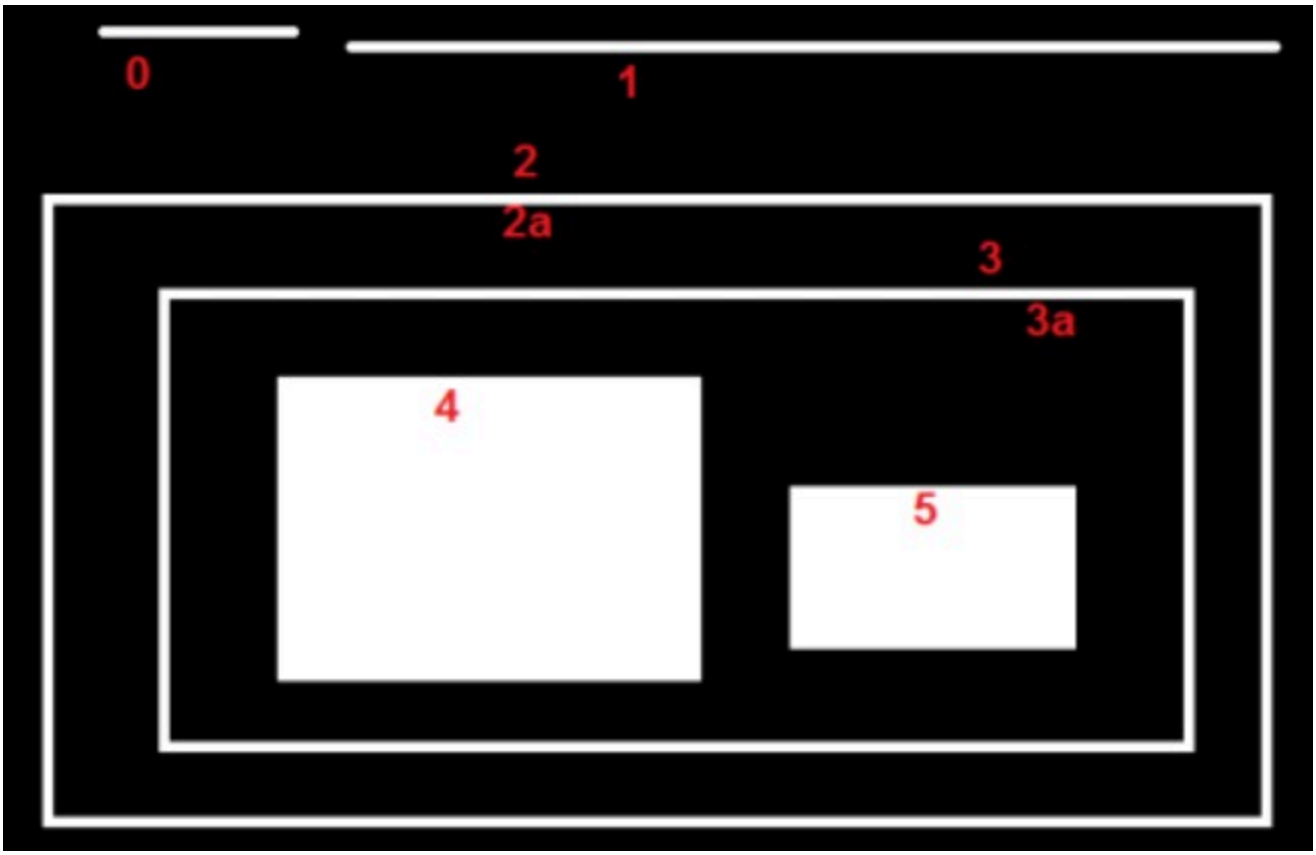
- 取出所有轮廓，并建立完整多层树状结构（谁是爷爷、爸爸、儿子、孙子…）。
- 保留所有嵌套关系。
- 适用场景：复杂嵌套分析、需要知道精确包含关系时（推荐学习阶段使用）

Hierarchy 的结构（最核心内容）

hierarchy 是一个 NumPy 数组，形状通常是 (1, N, 4)，其中 N 是轮廓数量。对于每一个轮廓 i，hierarchy[0][i] 包含 4 个整数：

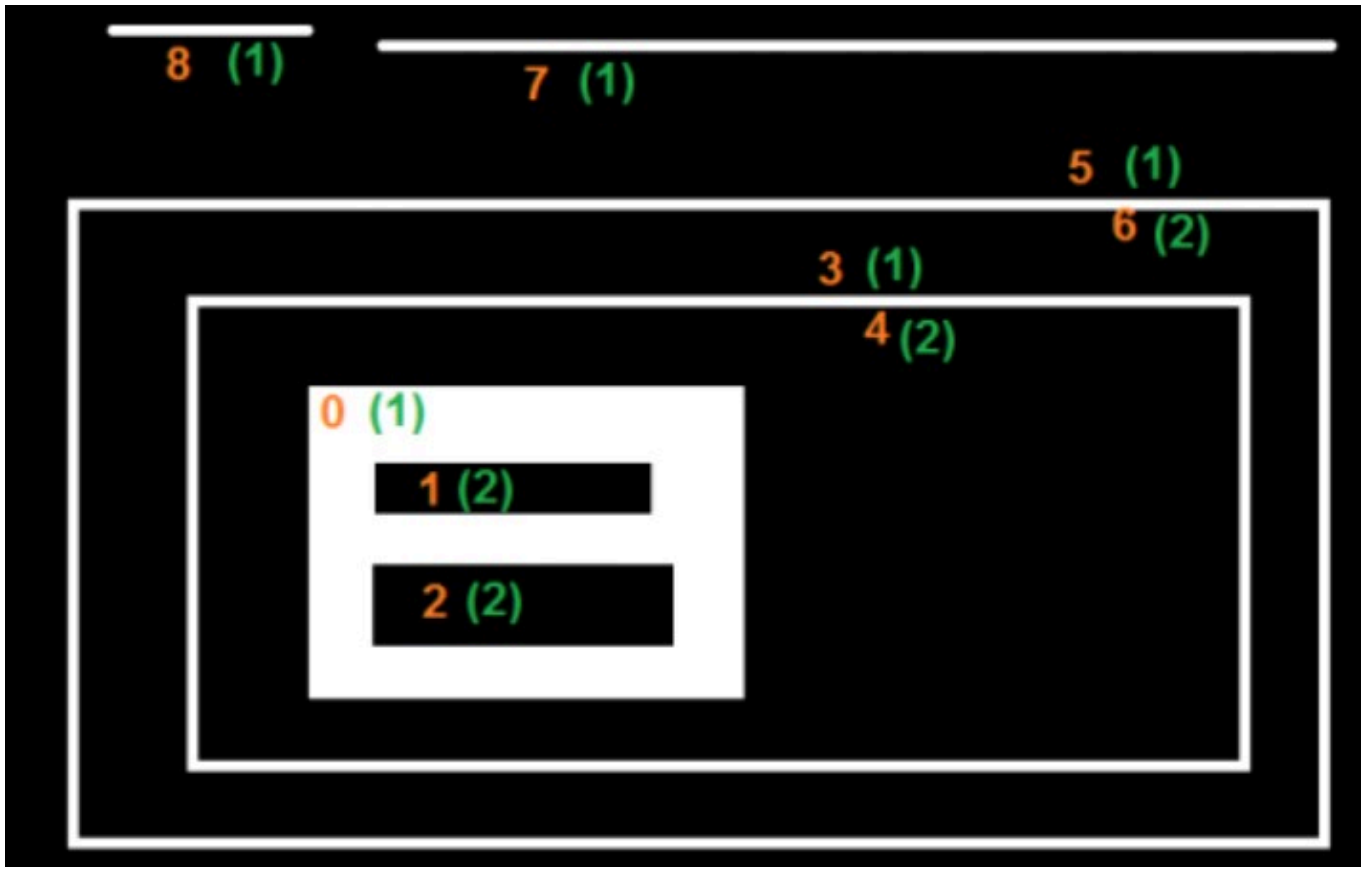
```
[Next, Previous, First_Child, Parent]
```

- Next：同一层级中下一个轮廓的索引（-1 表示没有下一个）
- Previous：同一层级中上一个轮廓的索引（-1 表示没有上一个）
- First_Child：它的第一个子轮廓索引（-1 表示没有孩子）
- Parent：它的父轮廓索引（-1 表示没有父亲，是最外层）



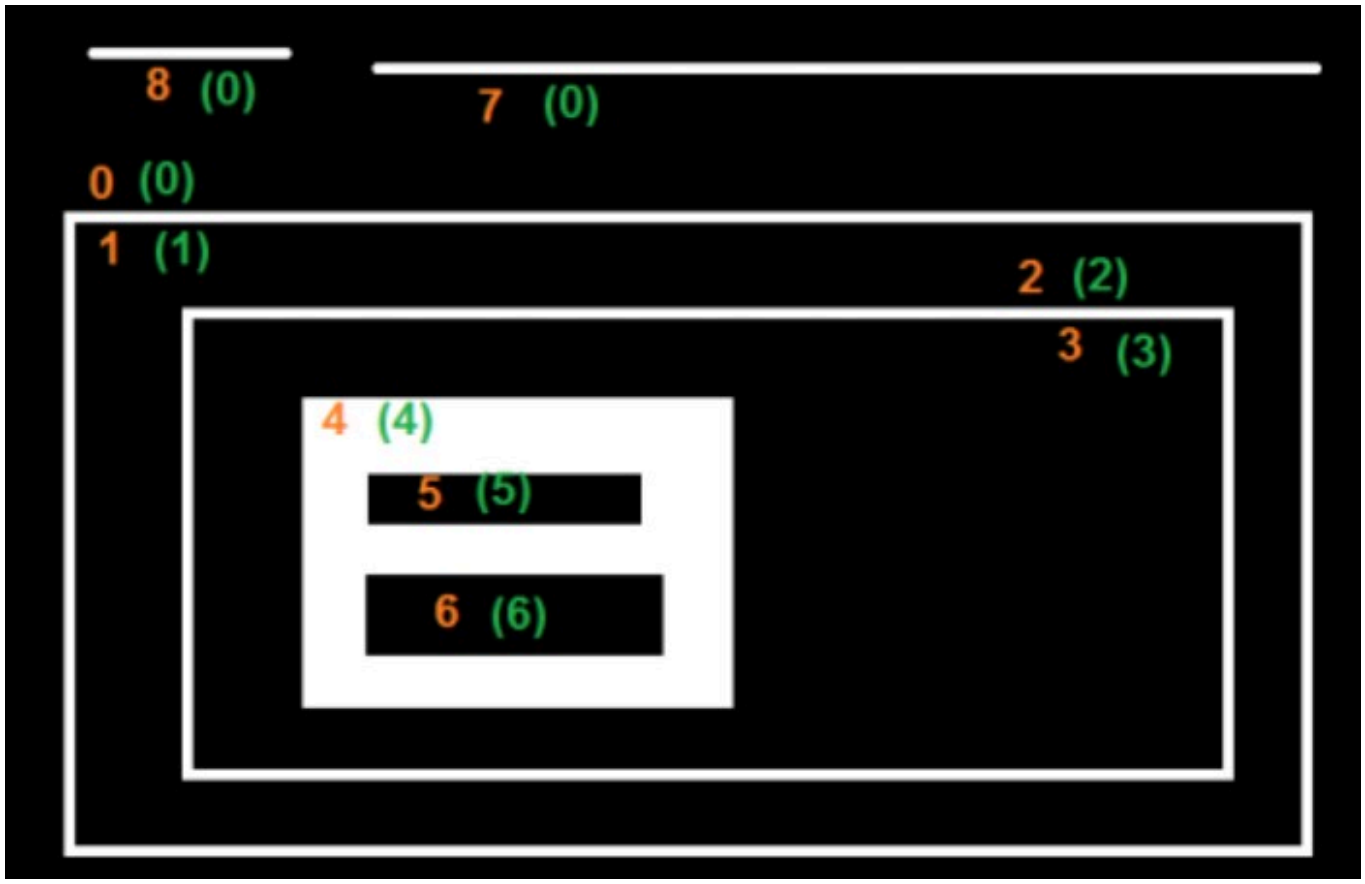
```
1. cv.RETR_LIST
>>> hierarchy
array([[[ 1, -1, -1, -1],
        [ 2,  0, -1, -1],
        [ 3,  1, -1, -1],
        [ 4,  2, -1, -1],
        [ 5,  3, -1, -1],
        [ 6,  4, -1, -1],
        [ 7,  5, -1, -1],
        [-1,  6, -1, -1]]])

2. cv.RETR_EXTERNAL
>>> hierarchy
array([[[ 1, -1, -1, -1],
        [ 2,  0, -1, -1],
        [-1,  1, -1, -1]]])
```




```
3. cv.RETR_CCOMP
>>> hierarchy
array([[[ 3, -1,  1, -1],
        [ 2, -1, -1,  0],
        [-1,  1, -1,  0],
        [ 5,  0,  4, -1],
```

```
        [-1, -1, -1,  3],
        [ 7,  3,  6, -1],
        [-1, -1, -1,  5],
        [ 8,  5, -1, -1],
        [-1,  7, -1, -1]]])
```




```
4. cv.RETR_TREE
>>> hierarchy
array([[[ 7, -1,  1, -1],
        [-1, -1,  2,  0],
        [-1, -1,  3,  1],
        [-1, -1,  4,  2],
        [-1, -1,  5,  3],
        [ 6, -1, -1,  4],
        [-1,  5, -1,  4],
        [ 8,  0, -1, -1],
        [-1,  7, -1, -1]]])
```